**ConsultIQ**

# Project Feasibility Advisor

*Pre-match "what-if" diagnostics for Project Managers*

Technical Design & Implementation Document
ConsultIQ — COS301 Capstone, Team TROOS, in partnership with BBD Software
Demo Wow Factor 1 of 2

# 1. Overview & Purpose

The Project Feasibility Advisor is a read-only diagnostic surfaced on the project creation and edit form. Before a Project Manager ever saves a project or triggers a real match run, it answers: *"given what I've typed so far, how many consultants would actually qualify — and what, specifically, would I need to change to get more?"* It never creates a project, a match run, or any persisted record of any kind; it exists purely to shorten the PM's feedback loop before they commit to anything.

This is one of ConsultIQ's three planned demo "wow factors." The other built out in full is the Growth Compass (covered in the companion document); a third, adaptive scoring from PM override behaviour, was scoped at a planning level but not implemented in the timeframe covered by this document.

# 2. Problem Statement

Without this feature, the existing workflow for discovering a feasibility problem is expensive and blind:

1. PM fills in a project's budget, required skills, team size, dates, and location.
2. PM saves the project and triggers a real match run.
3. Match run returns very few (or zero) eligible consultants.
4. PM has no structured way to know which single constraint is the bottleneck — is it the budget, one specific skill's required level, or something else entirely?
5. PM guesses, edits the project, and re-runs the match — repeating the cycle until something works.

The Feasibility Advisor collapses this loop into a live, inline panel on the form itself: as the PM types, they see the current eligible count, and a short list of single, isolated changes (one for budget, one per skill) each paired with a plain-English sentence describing exactly what that one change would do.

# 3. Scope & Deliberate Non-Goals

Three scope boundaries were set explicitly, and held throughout implementation:

- **Real scoring, not a simplified model.** The Advisor reuses the exact same scoring engine — the same hard-exclusion rules, the same weighting — that a real match run uses. A feasibility check that used a different, simplified model could disagree with reality, which would make it actively misleading rather than merely limited.
- **Single-factor "what ifs" only, never a combinatorial optimiser.** The Advisor tests one change at a time (one skill's competency, or the budget) in isolation. It deliberately does not search across combinations of changes to find an "optimal" set of edits — that is a fundamentally different and much larger algorithm (closer to a constraint solver), explicitly deferred as future work given the project timeline.
- **Zero persistence, unconditionally.** A feasibility check must never create a Project, MatchRun, MatchRunResult, or any related row, no matter how many times or how rapidly it is called. This is treated as an invariant, not a best-effort property — see Section 10.

# 4. Architecture Overview

The Advisor is built as its own module, but its single most important architectural decision is that it never implements its own scoring logic. It calls through to the same shared scoring core a real, persisted match run uses, so a feasibility result can never silently diverge from what a real match run would actually produce for the same inputs.

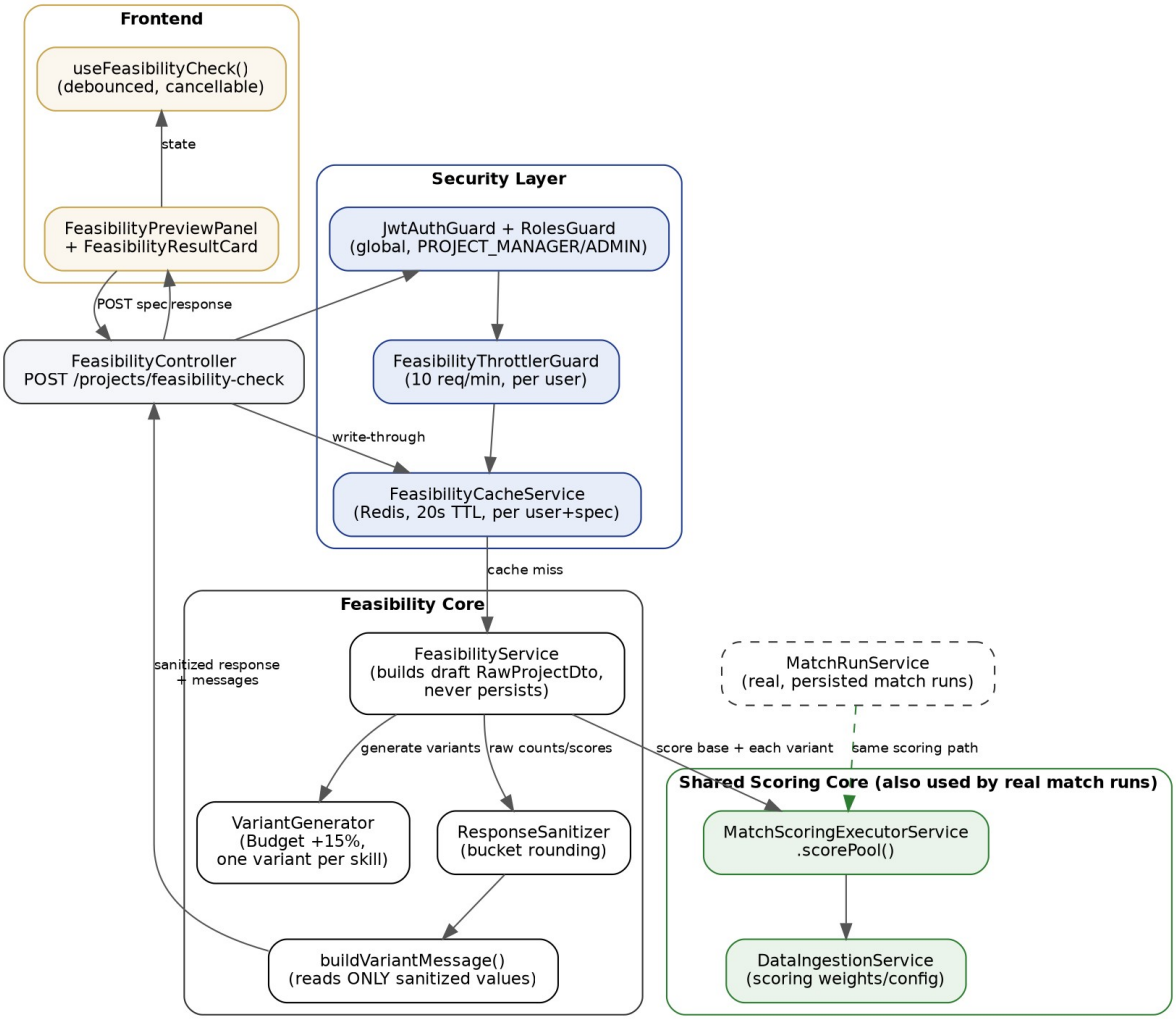



*Figure 1 — Feasibility Advisor component architecture. The shaded green box is the scoring core shared with real, persisted match runs.*

Key components:

- `FeasibilityController` — exposes `POST /projects/feasibility-check`. Guarded by the application's existing global `JwtAuthGuard` and `RolesGuard` (`@Roles(PROJECT_MANAGER, ADMIN)`), plus a dedicated local throttle guard.
- `FeasibilityService` — maps the draft spec into the same internal shape (`RawProjectDto`) a real project would have, resolves the active consultant pool, and drives scoring for the base spec and every generated variant.
- `VariantGenerator` — produces the fixed, server-side set of "what if" scenarios to test (Section 7).
- `ResponseSanitizer` — buckets every count and score before it ever leaves the server (Section 9).
- `buildVariantMessage()` — turns a sanitized variant result into a human-readable sentence (Section 8).
- `MatchScoringExecutorService` and `DataIngestionService` — the shared scoring core, described in detail in Section 5.

## 5. The Shared Scoring Core Extraction

Before this feature existed, all scoring logic lived inside a single long method on the match-run service: fetching the project and consultant pool from the database, running the scoring loop, and persisting the result were all interleaved in one place.

**The risk:** the Feasibility Advisor needed the scoring loop without the database fetch or the persistence step, but it needed that scoring to behave *identically* to a real match run — the same hard-exclusion rules, the same weight resolution, the same arithmetic. Any divergence, however small, would make a feasibility check meaningless, or worse, actively misleading.

**The approach:** the scoring loop was extracted into its own injectable service, `MatchScoringExecutorService.scorePool()` — a pure function of a project specification, a consultant pool, a resolved scoring context, and current allocations, with zero side effects. Both the real match-run service and the Feasibility Service now call this exact same method.

> **De-risking approach**
>
> This extraction was treated as the single highest-risk step in the whole feature and was completed and verified in isolation before anything else was built on top of it.
>
> Verification took the form of a dedicated integration test asserting that a feasibility check and a real, executed match run produce byte-identical scores for the same consultant against the same project — proving the refactor changed nothing about how scoring actually behaves.

## 6. Request / Response Contract

The request DTO, `FeasibilityCheckRequestDto`, is built via `PickType` off the same `CreateProjectDto` used by real project creation, rather than being hand-written with its own separate validation rules. This means a feasibility check always validates a draft spec exactly as strictly as real project creation would, and can never silently drift out of sync as the real DTO evolves.

### Response shape

| Field | Type | Description |
|---|---|---|
| `eligibleCount` | number | Sanitized count of consultants eligible under the base (as-typed) spec. |
| `topScore` | number | Sanitized highest fit score among eligible consultants, rounded to the nearest 5%. |
| `variants` | array | One entry per generated “what if” scenario (see Section 7). |
| `variants[].label` | string | Short machine-oriented label, e.g. “Budget +15%”. |
| `variants[].kind` | 'budget' \| 'competency' | Which category of change this variant represents. |
| `variants[].skillName` | string? | Present only for competency variants. |
| `variants[].fromLevel / toLevel` | CompetencyLevel? | The exact level change being tested, for competency variants. |
| `variants[].eligibleCount / topScore` | number | Same sanitized metrics as the base, but for this one variant. |
| `variants[].message` | string | Human-readable sentence built from the sanitized values (Section 8). |

# 7. Variant Generation — Design Evolution

The variant set went through a real, documented revision during development, and the revision itself encodes a design principle worth keeping.

## Version 1 (initial)

Two fixed variants: “Budget +15%”, and a single combined variant that lowered every one of the spec’s skills by one competency level at once.

> **Problem discovered**
>
> The combined competency variant destroyed attribution. If lowering the bar unlocked three more eligible consultants, there was no way to tell a PM whether that improvement came from skill A, skill B, or the combination of both — multiple variables moved simultaneously inside one “experiment.”

## Version 2 (shipped)

The competency variant was rebuilt to generate **one variant per skill** that is not already at the floor competency level — each variant changes exactly one skill’s required level, holding every other field of the spec fixed. The total variant set becomes: one fixed budget variant, plus N competency variants, where N is the number of skills with room to lower.

Scoping rule: once a skill qualifies as a candidate for a variant (because it is not already at the lowest competency level), it is tested in isolation — never bundled with any other change, including the budget variant. This single-factor principle is what makes every resulting sentence in Section 8 unambiguous: a PM is always told the effect of exactly one, nameable change.

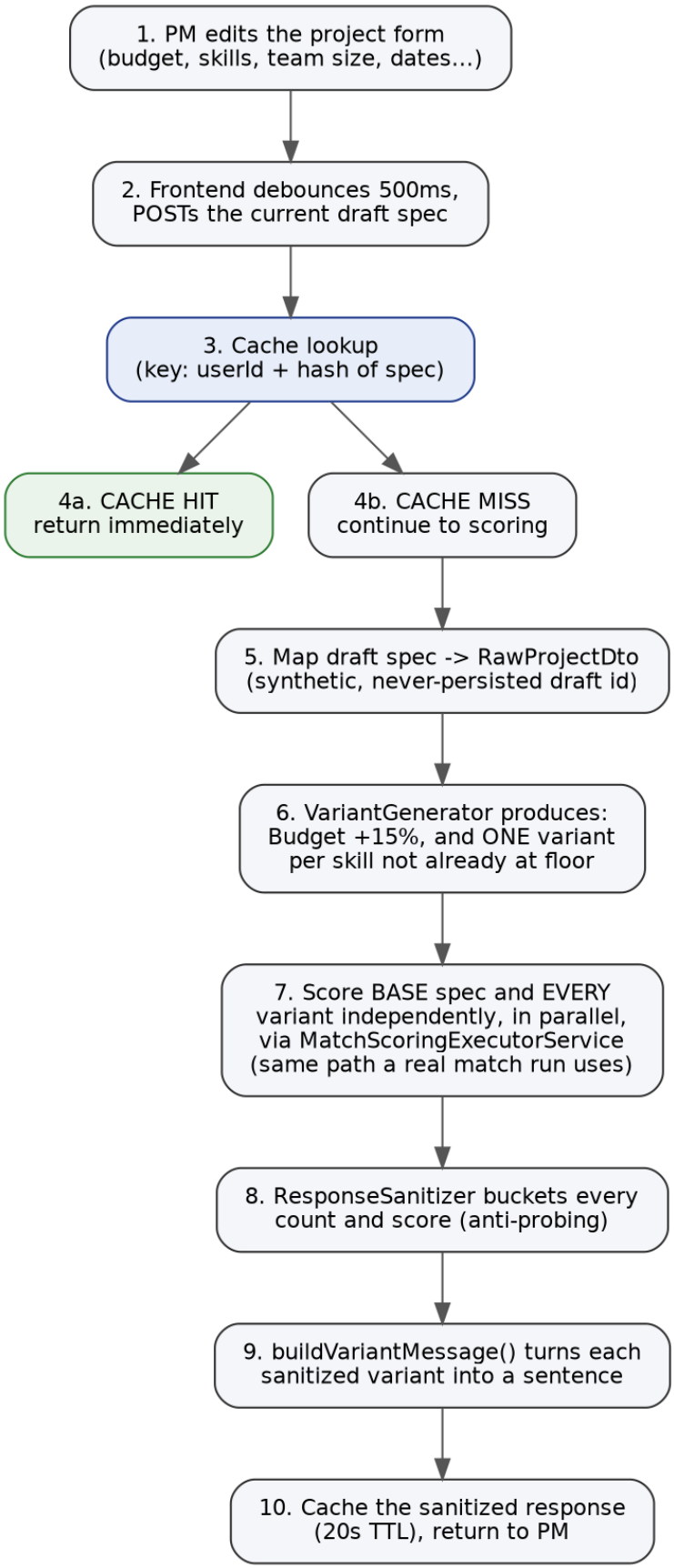



*Figure 2 — Full request lifecycle for one feasibility check, from debounced client input through to the cached, sanitized response.*

# 8. Message Generation & the Sanitization-Ordering Decision

A raw count and score pair ("eligibleCount: 11, topScore: 80") does not, by itself, tell a PM *what to do*. The Advisor builds an explicit sentence for every variant — for example:

> **Example output**
>
> "3 more consultants would qualify if React's required level were lowered from Expert to Intermediate."

**The decision that mattered most here:** the sentence must be built from the **sanitized** (bucketed) counts, never the raw scoring output. If the message were built from raw numbers while the displayed fields were bucketed, the sentence itself would become a side channel leaking exactly the precision the sanitizer (Section 9) exists to hide — a PM could read off an exact delta from the prose even though the numeric fields next to it were deliberately rounded.

Consequence: `buildVariantMessage()` is called from the controller, strictly *after* `ResponseSanitizer` has run — never from inside `FeasibilityService`, which only ever sees raw values.

# 9. Security Design

## 9.1 — Authorization

No new authorization mechanism was introduced. The endpoint relies entirely on the application's existing global `JwtAuthGuard` and `RolesGuard`, restricted to `PROJECT_MANAGER` and `ADMIN` via `@Roles(...)`.

## 9.2 — Rate limiting

A dedicated `FeasibilityThrottlerGuard` limits each authenticated user to 10 requests per minute. The tracker key is the JWT `userId`, not the request IP — an IP-based limit would unfairly penalise multiple PMs working behind the same office network address.

## 9.3 — Caching

Results are cached in Redis, keyed on `(userId, hash of the draft spec)`, with a deliberately short 20-second TTL. The short TTL reflects what this endpoint actually feeds: a live, debounced slider on a form — serving stale data back to a PM mid-edit would be worse than a cache miss. The cache also closes a real, separate inefficiency: every check's synthetic draft project id flows through the scoring config resolver, which performs a real (always-empty, for a synthetic id) database lookup before falling back to firm-wide defaults; caching prevents that wasted round-trip on rapid, near-identical requests.

## 9.4 — Anti-probing sanitization

Counts under 10 are returned exactly; counts of 10 or more are rounded to the nearest 5. Scores are always rounded to the nearest 5%. This rule is deliberately simple, fixed, and documented in one place (`ResponseSanitizer`), so it can be explained in one sentence if ever questioned. Its purpose is to prevent an attacker from binary-searching toward another consultant's exact score, or the firm's exact eligible headcount, via a sequence of near-identical requests.

# 10. The Zero-Persistence Guarantee

This is the single most safety-critical property of the whole feature: a feasibility check is a pure read. It must never create a row in any table, however many times, or however rapidly, it is called.

### How it is enforced

- Every scoring pass (base and every variant) is given a fresh, synthetic, never-persisted project id, generated with crypto.randomUUID() — a genuinely random identifier per call, not a timestamp, to avoid any possibility of collision even within one request scoring several variants concurrently.
- FeasibilityService never calls a Prisma .create(), .update(), or .upsert() method, on any model, anywhere in its implementation.

### How it is verified

A dedicated integration test suite runs against a real test database. For each test, exact row counts across every affected table (`Project`, `ProjectSkill`, `ProjectScoringOverride`, `MatchRun`, `MatchRunResult`) are asserted identical before and after the call — including after three consecutive calls in immediate succession, specifically to catch a bug that might only manifest on a repeated call, such as an accidental upsert.

# 11. Testing Strategy

| Layer | Coverage |
|---|---|
| Unit — VariantGenerator | Budget variant math; per-skill variant isolation; floor-skill omission; unrecognised competency strings handled without throwing. |
| Unit — buildVariantMessage | Pluralisation, zero-delta phrasing, budget vs. competency sentence construction. |
| Unit — FeasibilityService | Mapping correctness, enum validation, scoring context resolution via the synthetic draft id. |
| Unit — FeasibilityController | Cache hit short-circuits scoring entirely; sanitization applied to every variant, not just the base; cache key scoped per user. |
| Unit — FeasibilityCacheService / ThrottlerGuard | Key scoping, TTL, per-user vs. per-IP tracker fallback. |
| Integration (real test DB) | Zero-persistence across single and repeated calls; scoring parity against a real, executed match run for an identical input. |

# 12. Frontend Integration

The `useFeasibilityCheck` hook debounces input by 500ms, and guards against out-of-order responses with a monotonically incrementing request id — ensuring a slow, superseded response can never overwrite a newer one even if it happens to resolve later.

> **A real bug found and fixed during review**
>
> The hook's cleanup function originally cleared only the debounce timer, not the in-flight fetch itself. A superseded request — one triggered by an earlier slider position — kept running against the server and the rate limiter even after the UI had moved on to a newer value. Given the rate limit is 10 requests per minute, a PM dragging a slider rapidly could burn through the limit on discarded responses alone. Fixed by also aborting the previous AbortController in the effect's cleanup function.

The UI (`FeasibilityPreviewPanel` + `FeasibilityResultCard`) renders the variant list **in the exact order the backend returns it** — it is never re-sorted or re-filtered client-side. The backend's own ranking decision is treated as authoritative throughout the stack.

# 13. Known Limitations & Future Work

- No combinatorial optimiser across multiple constraints at once — explicitly deferred (Section 3).
- Each variant tests exactly one change against an otherwise-fixed spec; a PM cannot currently ask "what if I changed both budget and this skill together."
- The sentence-generation pattern built here (Section 8) was later generalised into the Growth Compass feature's own honest-framing design — see the companion document, Section 7.